I cannot directly generate and serve a binary `.pdf` file through this chat interface. However, below is the **complete, consolidated source code** for the entire project, organized by file path. **To create your own PDF:** 1. Copy all the text below. 2. Paste it into any text editor (VS Code, Notepad++, etc.) or a Markdown editor. 3. Save it as `InsuranceClaimsSystem.md` or `.txt`. 4. Use **File → Print → Save as PDF** (or any Markdown-to-PDF converter). --- ```text
==========
========== FILE: InsuranceClaimsProcessingSystem.csproj
==========
========== <Project Sdk="Microsoft.NET.Sdk.Web"> <PropertyGroup>
<TargetFramework>net8.0</TargetFramework> <Nullable>enable</Nullable>
<ImplicitUsings>enable</ImplicitUsings> </PropertyGroup> <ItemGroup>
<PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.0"
/> <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.0" />
<PackageReference Include="Microsoft.AspNetCore.Identity.EntityFrameworkCore"
Version="8.0.0" /> </ItemGroup> </Project>
==========
========== FILE: Models/Enums/ClaimStatus.cs
==========
========== namespace InsuranceClaimsProcessingSystem.Models.Enums; public enum
ClaimStatus { REGISTERED, UNDER_REVIEW, APPROVED, REJECTED }
==========
========== FILE: Models/Enums/VerificationStatus.cs
==========
========== namespace InsuranceClaimsProcessingSystem.Models.Enums; public enum
VerificationStatus { PENDING, VERIFIED, REJECTED }
==========
========== FILE: Models/Enums/RiskFlag.cs
==========
========== namespace InsuranceClaimsProcessingSystem.Models.Enums; public enum
RiskFlag { LOW, MEDIUM, HIGH }
==========
========== FILE: Models/Claim.cs
==========
========== using System.ComponentModel.DataAnnotations; using
System.ComponentModel.DataAnnotations.Schema; using
InsuranceClaimsProcessingSystem.Models.Enums; namespace
InsuranceClaimsProcessingSystem.Models; public class Claim { [Key] public int claimId
{ get; set; } [Required] public int policyId { get; set; } [Required] public int customerId { get;

set; } [Required] [StringLength(50)] public string claimType { get; set; } = string.Empty; [Column(TypeName = "decimal(15,2)")] public decimal claimAmount { get; set; } public ClaimStatus claimStatus { get; set; } = ClaimStatus.REGISTERED; [ForeignKey("policyId")] public Policy? Policy { get; set; } [ForeignKey("customerId")] public Customer? Customer { get; set; } public ICollection<ClaimDocument> Documents { get; set; } = new List<ClaimDocument>(); public Assessment? Assessment { get; set; } public FraudCheck? FraudCheck { get; set; } public ICollection<SettlementLog> SettlementLogs { get; set; } = new List<SettlementLog>(); }
==========
========== FILE: Models/ClaimDocument.cs
==========
========== using System.ComponentModel.DataAnnotations; using InsuranceClaimsProcessingSystem.Models.Enums; namespace InsuranceClaimsProcessingSystem.Models; public class ClaimDocument { [Key] public int documentId { get; set; } [Required] public int claimId { get; set; } [Required] [StringLength(100)] public string documentName { get; set; } = string.Empty; [Required] [StringLength(255)] public string filePath { get; set; } = string.Empty; public VerificationStatus verificationStatus { get; set; } = VerificationStatus.PENDING; [ForeignKey("claimId")] public Claim? Claim { get; set; } }
==========
========== FILE: Models/Assessment.cs
==========
========== using System.ComponentModel.DataAnnotations; using System.ComponentModel.DataAnnotations.Schema; namespace InsuranceClaimsProcessingSystem.Models; public class Assessment { [Key] public int assessmentId { get; set; } [Required] public int claimId { get; set; } [Required] public int surveyorId { get; set; } [Column(TypeName = "decimal(15,2)")] public decimal assessedAmount { get; set; } [StringLength(255)] public string assessmentRemarks { get; set; } = string.Empty; [ForeignKey("claimId")] public Claim? Claim { get; set; } }
==========
========== FILE: Models/FraudCheck.cs
==========
========== using System.ComponentModel.DataAnnotations; using InsuranceClaimsProcessingSystem.Models.Enums; namespace InsuranceClaimsProcessingSystem.Models; public class FraudCheck { [Key] public int fraudId { get; set; } [Required] public int claimId { get; set; } public int fraudScore { get; set; } public RiskFlag riskFlag { get; set; } [ForeignKey("claimId")] public Claim? Claim { get; set; } }
==========

========== FILE: Models/SettlementLog.cs ==========
========== using System.ComponentModel.DataAnnotations; using System.ComponentModel.DataAnnotations.Schema; namespace InsuranceClaimsProcessingSystem.Models; public class SettlementLog { [Key] public int settlementId { get; set; } [Required] public int claimId { get; set; } [Column(TypeName = "decimal(15,2)")] public decimal settlementAmount { get; set; } [Required] [StringLength(50)] public string settlementStatus { get; set; } = string.Empty; [StringLength(255)] public string remarks { get; set; } = string.Empty; [ForeignKey("claimId")] public Claim? Claim { get; set; } }
========== FILE: Models/Customer.cs ==========
========== using System.ComponentModel.DataAnnotations; namespace InsuranceClaimsProcessingSystem.Models; public class Customer { [Key] public int customerId { get; set; } [Required] [StringLength(100)] public string customerName { get; set; } = string.Empty; [EmailAddress] [StringLength(100)] public string email { get; set; } = string.Empty; [Phone] [StringLength(20)] public string phone { get; set; } = string.Empty; public ICollection<Claim> Claims { get; set; } = new List<Claim>(); }
========== FILE: Models/Policy.cs ==========
========== using System.ComponentModel.DataAnnotations; using System.ComponentModel.DataAnnotations.Schema; namespace InsuranceClaimsProcessingSystem.Models; public class Policy { [Key] public int policyId { get; set; } [Required] [StringLength(100)] public string policyNumber { get; set; } = string.Empty; [Required] public int customerId { get; set; } [Column(TypeName = "decimal(15,2)")] public decimal coverageAmount { get; set; } public bool isActive { get; set; } = true; [ForeignKey("customerId")] public Customer? Customer { get; set; } public ICollection<Claim> Claims { get; set; } = new List<Claim>(); }
========== FILE: Models/ViewModels/ClaimDocumentUploadViewModel.cs ==========
========== using System.ComponentModel.DataAnnotations; namespace InsuranceClaimsProcessingSystem.Models.ViewModels; public class ClaimDocumentUploadViewModel { public int claimId { get; set; } [Required(ErrorMessage = "Document name required")] public string documentName { get; set; } = string.Empty; [Required(ErrorMessage = "File selection required")] public IFormFile documentFile { get; set; } = null!; }

==========
========== FILE: Models/ViewModels/AssignSurveyorViewModel.cs
==========
========== using System.ComponentModel.DataAnnotations; namespace InsuranceClaimsProcessingSystem.Models.ViewModels; public class AssignSurveyorViewModel { public int claimId { get; set; } [Required] public int surveyorId { get; set; } }
==========
========== FILE: Models/ViewModels/RecordAssessmentViewModel.cs
==========
========== using System.ComponentModel.DataAnnotations; namespace InsuranceClaimsProcessingSystem.Models.ViewModels; public class RecordAssessmentViewModel { public int claimId { get; set; } [Required] public int surveyorId { get; set; } [Required] public decimal assessedAmount { get; set; } public string? assessmentRemarks { get; set; } }
==========
========== FILE: Models/ViewModels/SettlementActionViewModel.cs
==========
========== using System.ComponentModel.DataAnnotations; namespace InsuranceClaimsProcessingSystem.Models.ViewModels; public class SettlementActionViewModel { public int claimId { get; set; } [Required] public decimal amount { get; set; } public string? remarks { get; set; } }
==========
========== FILE: Data/ApplicationDbContext.cs
==========
========== using InsuranceClaimsProcessingSystem.Models; using InsuranceClaimsProcessingSystem.Models.Enums; using Microsoft.AspNetCore.Identity.EntityFrameworkCore; using Microsoft.EntityFrameworkCore; namespace InsuranceClaimsProcessingSystem.Data; public class ApplicationDbContext : IdentityDbContext { public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options) { } public DbSet<Claim> Claims => Set<Claim>(); public DbSet<ClaimDocument> ClaimDocuments => Set<ClaimDocument>(); public DbSet<Assessment> Assessments => Set<Assessment>(); public DbSet<FraudCheck> FraudChecks => Set<FraudCheck>(); public DbSet<SettlementLog> SettlementLogs => Set<SettlementLog>(); public DbSet<Customer> Customers => Set<Customer>(); public DbSet<Policy> Policies => Set<Policy>(); protected override void OnModelCreating(ModelBuilder builder) { base.OnModelCreating(builder); builder.Entity<Claim>() .Property(c => c.claimStatus) .HasConversion<string>() .HasMaxLength(20);

builder.Entity<ClaimDocument>() .Property(d => d.verificationStatus) .HasConversion<string>() .HasMaxLength(20); builder.Entity<FraudCheck>() .Property(f => f.riskFlag) .HasConversion<string>() .HasMaxLength(10); builder.Entity<Claim>() .HasOne(c => c.Policy) .WithMany(p => p.Claims) .HasForeignKey(c => c.policyId) .OnDelete(DeleteBehavior.Restrict); builder.Entity<Claim>() .HasOne(c => c.Customer) .WithMany(cu => cu.Claims) .HasForeignKey(c => c.customerId) .OnDelete(DeleteBehavior.Cascade); builder.Entity<ClaimDocument>() .HasOne(d => d.Claim) .WithMany(c => c.Documents) .HasForeignKey(d => d.claimId) .OnDelete(DeleteBehavior.Cascade); builder.Entity<Assessment>() .HasOne(a => a.Claim) .WithOne(c => c.Assessment) .HasForeignKey<Assessment>(a => a.claimId); builder.Entity<FraudCheck>() .HasOne(f => f.Claim) .WithOne(c => c.FraudCheck) .HasForeignKey<FraudCheck>(f => f.claimId); builder.Entity<SettlementLog>() .HasOne(s => s.Claim) .WithMany(c => c.SettlementLogs) .HasForeignKey(s => s.claimId); } }
==========
========== FILE: Services/Interfaces/IClaimService.cs
==========
========== using InsuranceClaimsProcessingSystem.Models; namespace InsuranceClaimsProcessingSystem.Services.Interfaces; public interface IClaimService { Task<List<Claim>> getAllClaims(); Task<Claim?> getClaimDetails(int claimId); Task<Claim> registerClaim(Claim claim); Task<bool> updateClaim(Claim claim); Task<bool> validatePolicy(int claimId); }
==========
========== FILE: Services/Interfaces/IClaimDocumentService.cs
==========
========== using InsuranceClaimsProcessingSystem.Models; namespace InsuranceClaimsProcessingSystem.Services.Interfaces; public interface IClaimDocumentService { Task<ClaimDocument> uploadClaimDocument(ClaimDocument doc); Task<bool> validateClaimDocument(int documentId); Task<List<ClaimDocument>> getClaimDocuments(int claimId); }
==========
========== FILE: Services/Interfaces/IAssessmentService.cs
==========
========== using InsuranceClaimsProcessingSystem.Models; namespace InsuranceClaimsProcessingSystem.Services.Interfaces; public interface IAssessmentService { Task assignSurveyor(int claimId, int surveyorId); Task recordAssessment(Assessment assessment); Task<Assessment?> getAssessmentDetails(int claimId); }

==========
========== FILE: Services/Interfaces/IFraudService.cs
==========
========== using InsuranceClaimsProcessingSystem.Models; namespace InsuranceClaimsProcessingSystem.Services.Interfaces; public interface IFraudService { Task<FraudCheck> runFraudCheck(int claimId); Task<FraudCheck?> getFraudScore(int claimId); }
==========
========== FILE: Services/Interfaces/ISettlementService.cs
==========
========== using InsuranceClaimsProcessingSystem.Models; namespace InsuranceClaimsProcessingSystem.Services.Interfaces; public interface ISettlementService { Task<SettlementLog> approveSettlement(SettlementLog log); Task<SettlementLog> rejectSettlement(SettlementLog log); Task<bool> processPayment(int settlementId); }
==========
========== FILE: Services/ClaimService.cs
==========
========== using InsuranceClaimsProcessingSystem.Data; using InsuranceClaimsProcessingSystem.Models; using InsuranceClaimsProcessingSystem.Models.Enums; using InsuranceClaimsProcessingSystem.Services.Interfaces; using Microsoft.EntityFrameworkCore; namespace InsuranceClaimsProcessingSystem.Services; public class ClaimService : IClaimService { private readonly ApplicationDbContext _context; public ClaimService(ApplicationDbContext context) { _context = context; } public async Task<List<Claim>> getAllClaims() { return await _context.Claims .Include(c => c.Policy) .Include(c => c.Customer) .Include(c => c.FraudCheck) .Include(c => c.Assessment) .ToListAsync(); } public async Task<Claim?> getClaimDetails(int claimId) { return await _context.Claims .Include(c => c.Policy) .Include(c => c.Customer) .Include(c => c.Documents) .Include(c => c.Assessment) .Include(c => c.FraudCheck) .Include(c => c.SettlementLogs.OrderByDescending(s => s.settlementId)) .FirstOrDefaultAsync(c => c.claimId == claimId); } public async Task<Claim> registerClaim(Claim claim) { claim.claimStatus = ClaimStatus.REGISTERED; _context.Claims.Add(claim); await _context.SaveChangesAsync(); return claim; } public async Task<bool> updateClaim(Claim claim) { var existing = await _context.Claims.FindAsync(claim.claimId); if (existing == null) return false; existing.policyId = claim.policyId; existing.customerId = claim.customerId; existing.claimType = claim.claimType; existing.claimAmount = claim.claimAmount; await _context.SaveChangesAsync(); return true; } public async Task<bool> validatePolicy(int claimId) { var claim = await _context.Claims .Include(c => c.Policy) .FirstOrDefaultAsync(c => c.claimId == claimId); if (claim?.Policy == null || !claim.Policy.isActive) { if (claim != null)

{ claim.claimStatus = ClaimStatus.REJECTED; await _context.SaveChangesAsync(); } return false; } if (claim.claimAmount > claim.Policy.coverageAmount) { claim.claimStatus = ClaimStatus.REJECTED; await _context.SaveChangesAsync(); return false; } claim.claimStatus = ClaimStatus.UNDER_REVIEW; await _context.SaveChangesAsync(); return true; } }
==========
========== FILE: Services/ClaimDocumentService.cs
==========
========== using InsuranceClaimsProcessingSystem.Data; using InsuranceClaimsProcessingSystem.Models; using InsuranceClaimsProcessingSystem.Models.Enums; using InsuranceClaimsProcessingSystem.Services.Interfaces; using Microsoft.EntityFrameworkCore; namespace InsuranceClaimsProcessingSystem.Services; public class ClaimDocumentService : IClaimDocumentService { private readonly ApplicationDbContext _context; public ClaimDocumentService(ApplicationDbContext context) { _context = context; } public async Task<ClaimDocument> uploadClaimDocument(ClaimDocument doc) { _context.ClaimDocuments.Add(doc); await _context.SaveChangesAsync(); return doc; } public async Task<bool> validateClaimDocument(int documentId) { var doc = await _context.ClaimDocuments.FindAsync(documentId); if (doc == null) return false; await Task.Delay(500); doc.verificationStatus = VerificationStatus.VERIFIED; await _context.SaveChangesAsync(); return true; } public async Task<List<ClaimDocument>> getClaimDocuments(int claimId) { return await _context.ClaimDocuments .Where(d => d.claimId == claimId) .ToListAsync(); } }
==========
========== FILE: Services/AssessmentService.cs
==========
========== using InsuranceClaimsProcessingSystem.Data; using InsuranceClaimsProcessingSystem.Models; using InsuranceClaimsProcessingSystem.Services.Interfaces; using Microsoft.EntityFrameworkCore; namespace InsuranceClaimsProcessingSystem.Services; public class AssessmentService : IAssessmentService { private readonly ApplicationDbContext _context; public AssessmentService(ApplicationDbContext context) { _context = context; } public async Task assignSurveyor(int claimId, int surveyorId) { var assessment = await _context.Assessments.FirstOrDefaultAsync(a => a.claimId == claimId); if (assessment != null) { assessment.surveyorId = surveyorId; } else { _context.Assessments.Add(new Assessment { claimId = claimId, surveyorId = surveyorId, assessedAmount = 0 }); } await _context.SaveChangesAsync(); } public async Task recordAssessment(Assessment assessment) { var existing = await

_context.Assessments.FirstOrDefaultAsync(a => a.claimId == assessment.claimId); if (existing != null) { existing.surveyorId = assessment.surveyorId; existing.assessedAmount = assessment.assessedAmount; existing.assessmentRemarks = assessment.assessmentRemarks; } else { _context.Assessments.Add(assessment); } await _context.SaveChangesAsync(); } public async Task<Assessment?> getAssessmentDetails(int claimId) { return await _context.Assessments.FirstOrDefaultAsync(a => a.claimId == claimId); } }
==========
========== FILE: Services/FraudService.cs
==========
========== using InsuranceClaimsProcessingSystem.Data; using InsuranceClaimsProcessingSystem.Models; using InsuranceClaimsProcessingSystem.Models.Enums; using InsuranceClaimsProcessingSystem.Services.Interfaces; using Microsoft.EntityFrameworkCore; namespace InsuranceClaimsProcessingSystem.Services; public class FraudService : IFraudService { private readonly ApplicationDbContext _context; public FraudService(ApplicationDbContext context) { _context = context; } public async Task<FraudCheck> runFraudCheck(int claimId) { var claim = await _context.Claims .Include(c => c.Policy) .Include(c => c.Documents) .FirstOrDefaultAsync(c => c.claimId == claimId); if (claim == null) throw new Exception("Claim not found"); int score = 0; if (claim.Policy != null && claim.claimAmount > (claim.Policy.coverageAmount * 0.8m)) score += 40; if (!claim.Documents.Any()) score += 30; if (claim.claimAmount > 500000m) score += 30; RiskFlag flag; if (score >= 70) flag = RiskFlag.HIGH; else if (score >= 40) flag = RiskFlag.MEDIUM; else flag = RiskFlag.LOW; var existing = await _context.FraudChecks.FirstOrDefaultAsync(f => f.claimId == claimId); if (existing != null) { existing.fraudScore = score; existing.riskFlag = flag; } else { _context.FraudChecks.Add(new FraudCheck { claimId = claimId, fraudScore = score, riskFlag = flag }); } await _context.SaveChangesAsync(); return existing ?? new FraudCheck { claimId = claimId, fraudScore = score, riskFlag = flag }; } public async Task<FraudCheck?> getFraudScore(int claimId) { return await _context.FraudChecks.FirstOrDefaultAsync(f => f.claimId == claimId); } }
==========
========== FILE: Services/SettlementService.cs
==========
========== using InsuranceClaimsProcessingSystem.Data; using InsuranceClaimsProcessingSystem.Models; using InsuranceClaimsProcessingSystem.Models.Enums; using InsuranceClaimsProcessingSystem.Services.Interfaces; using Microsoft.EntityFrameworkCore; namespace InsuranceClaimsProcessingSystem.Services;

public class SettlementService : ISettlementService { private readonly ApplicationDbContext _context; public SettlementService(ApplicationDbContext context) { _context = context; } public async Task<SettlementLog> approveSettlement(SettlementLog log) { log.settlementStatus = "APPROVED"; var claim = await _context.Claims.FindAsync(log.claimId); if (claim != null) claim.claimStatus = ClaimStatus.APPROVED; _context.SettlementLogs.Add(log); await _context.SaveChangesAsync(); return log; } public async Task<SettlementLog> rejectSettlement(SettlementLog log) { log.settlementStatus = "REJECTED"; log.settlementAmount = 0; var claim = await _context.Claims.FindAsync(log.claimId); if (claim != null) claim.claimStatus = ClaimStatus.REJECTED; _context.SettlementLogs.Add(log); await _context.SaveChangesAsync(); return log; } public async Task<bool> processPayment(int settlementId) { var log = await _context.SettlementLogs.FindAsync(settlementId); if (log == null || log.settlementStatus != "APPROVED") return false; log.settlementStatus = "PAYMENT_PROCESSED"; log.remarks += " | Payment disbursed via simulated gateway."; await _context.SaveChangesAsync(); return true; } }
==========
========== FILE: Controllers/ClaimController.cs
==========
========== using InsuranceClaimsProcessingSystem.Models; using InsuranceClaimsProcessingSystem.Services.Interfaces; using Microsoft.AspNetCore.Authorization; using Microsoft.AspNetCore.Mvc; namespace InsuranceClaimsProcessingSystem.Controllers; [Authorize] public class ClaimController : Controller { private readonly IClaimService _claimService; public ClaimController(IClaimService claimService) { _claimService = claimService; } public async Task<IActionResult> Index() { var claims = await _claimService.getAllClaims(); return View(claims); } public async Task<IActionResult> Details(int id) { var claim = await _claimService.getClaimDetails(id); if (claim == null) return NotFound(); return View(claim); } [Authorize(Roles = "Customer,ClaimsOfficer,Admin")] public IActionResult RegisterClaim() { return View(new Claim()); } [HttpPost] [Authorize(Roles = "Customer,ClaimsOfficer,Admin")] [ValidateAntiForgeryToken] public async Task<IActionResult> RegisterClaim(Claim claim) { if (!ModelState.IsValid) return View(claim); await _claimService.registerClaim(claim); TempData["Success"] = "Claim registered successfully!"; return RedirectToAction(nameof(Details), new { id = claim.claimId }); } [HttpPost] [Authorize(Roles = "ClaimsOfficer,Admin")] [ValidateAntiForgeryToken] public async Task<IActionResult> ValidatePolicy(int claimId) { bool isValid = await _claimService.validatePolicy(claimId); if (isValid) TempData["Success"] = "Policy validated. Status updated to UNDER_REVIEW."; else TempData["Error"] = "Policy validation failed. Claim rejected."; return

RedirectToAction(nameof(Details), new { id = claimId }); } }
==========
========== FILE: Controllers/ClaimDocumentController.cs
==========
========== using InsuranceClaimsProcessingSystem.Data; using InsuranceClaimsProcessingSystem.Models; using InsuranceClaimsProcessingSystem.Models.ViewModels; using InsuranceClaimsProcessingSystem.Services.Interfaces; using Microsoft.AspNetCore.Authorization; using Microsoft.AspNetCore.Mvc; namespace InsuranceClaimsProcessingSystem.Controllers; [Authorize] public class ClaimDocumentController : Controller { private readonly IClaimDocumentService _documentService; private readonly ApplicationDbContext _context; private readonly IWebHostEnvironment _environment; public ClaimDocumentController(IClaimDocumentService documentService, ApplicationDbContext context, IWebHostEnvironment environment) { _documentService = documentService; _context = context; _environment = environment; } [HttpGet] [Authorize(Roles = "Customer,ClaimsOfficer,Surveyor")] public IActionResult Upload(int claimId) { return View(new ClaimDocumentUploadViewModel { claimId = claimId }); } [HttpPost] [Authorize(Roles = "Customer,ClaimsOfficer,Surveyor")] [ValidateAntiForgeryToken] public async Task<IActionResult> UploadClaimDocument(ClaimDocumentUploadViewModel vm) { if (!ModelState.IsValid) return View(vm); var uploadsPath = Path.Combine(_environment.WebRootPath, "uploads"); Directory.CreateDirectory(uploadsPath); var uniqueFileName = Guid.NewGuid().ToString() + "_" + vm.documentFile.FileName; var fullPath = Path.Combine(uploadsPath, uniqueFileName); using (var fileStream = new FileStream(fullPath, FileMode.Create)) { await vm.documentFile.CopyToAsync(fileStream); } var document = new ClaimDocument { claimId = vm.claimId, documentName = vm.documentName, filePath = "/uploads/" + uniqueFileName, verificationStatus = Models.Enums.VerificationStatus.PENDING }; await _documentService.uploadClaimDocument(document); await _documentService.validateClaimDocument(document.documentId); TempData["Success"] = $"Document '{vm.documentName}' uploaded and verified."; return RedirectToAction("Details", "Claim", new { id = vm.claimId }); } }
==========
========== FILE: Controllers/AssessmentController.cs
==========
========== using InsuranceClaimsProcessingSystem.Models; using InsuranceClaimsProcessingSystem.Models.ViewModels; using InsuranceClaimsProcessingSystem.Services.Interfaces; using

Microsoft.AspNetCore.Authorization; using Microsoft.AspNetCore.Mvc; namespace InsuranceClaimsProcessingSystem.Controllers; [Authorize(Roles = "ClaimsOfficer,Surveyor,Admin")] public class AssessmentController : Controller { private readonly IAssessmentService _assessmentService; public AssessmentController(IAssessmentService assessmentService) { _assessmentService = assessmentService; } [HttpGet] public IActionResult AssignSurveyor(int claimId) { return View(new AssignSurveyorViewModel { claimId = claimId }); } [HttpPost] [ValidateAntiForgeryToken] public async Task<IActionResult> AssignSurveyor(AssignSurveyorViewModel vm) { if (!ModelState.IsValid) return View(vm); await _assessmentService.assignSurveyor(vm.claimId, vm.surveyorId); TempData["Success"] = $"Surveyor #{vm.surveyorId} assigned."; return RedirectToAction("Details", "Claim", new { id = vm.claimId }); } [HttpGet] public IActionResult RecordAssessment(int claimId) { return View(new RecordAssessmentViewModel { claimId = claimId }); } [HttpPost] [ValidateAntiForgeryToken] public async Task<IActionResult> RecordAssessment(RecordAssessmentViewModel vm) { if (!ModelState.IsValid) return View(vm); var assessment = new Assessment { claimId = vm.claimId, surveyorId = vm.surveyorId, assessedAmount = vm.assessedAmount, assessmentRemarks = vm.assessmentRemarks ?? "" }; await _assessmentService.recordAssessment(assessment); TempData["Success"] = "Assessment recorded."; return RedirectToAction("Details", "Claim", new { id = vm.claimId }); } }
==========
========== FILE: Controllers/FraudController.cs
==========
========== using InsuranceClaimsProcessingSystem.Models; using InsuranceClaimsProcessingSystem.Services.Interfaces; using Microsoft.AspNetCore.Authorization; using Microsoft.AspNetCore.Mvc; namespace InsuranceClaimsProcessingSystem.Controllers; [Authorize(Roles = "ComplianceOfficer,Admin")] public class FraudController : Controller { private readonly IFraudService _fraudService; public FraudController(IFraudService fraudService) { _fraudService = fraudService; } [HttpPost] [ValidateAntiForgeryToken] public async Task<IActionResult> RunFraudCheck(int claimId) { try { var result = await _fraudService.runFraudCheck(claimId); if (result.riskFlag == Models.Enums.RiskFlag.HIGH) TempData["Warning"] = $"HIGH RISK! Score: {result.fraudScore}"; else if (result.riskFlag == Models.Enums.RiskFlag.MEDIUM) TempData["Info"] = $"MEDIUM RISK. Score: {result.fraudScore}"; else TempData["Success"] = $"LOW RISK. Score: {result.fraudScore}"; } catch (Exception ex) { TempData["Error"] = ex.Message; } return RedirectToAction("Details", "Claim", new { id = claimId }); } [HttpGet] public async Task<IActionResult> GetFraudScore(int claimId) { var score = await

_fraudService.getFraudScore(claimId); if (score == null) return NotFound(); return View(score); } }
==========
========== FILE: Controllers/SettlementController.cs
==========
========== using InsuranceClaimsProcessingSystem.Models; using InsuranceClaimsProcessingSystem.Services.Interfaces; using Microsoft.AspNetCore.Authorization; using Microsoft.AspNetCore.Mvc; namespace InsuranceClaimsProcessingSystem.Controllers; [Authorize(Roles = "ClaimsOfficer,Admin")] public class SettlementController : Controller { private readonly ISettlementService _settlementService; public SettlementController(ISettlementService settlementService) { _settlementService = settlementService; } [HttpPost] [ValidateAntiForgeryToken] public async Task<IActionResult> ApproveSettlement([FromForm] SettlementActionViewModel vm) { if (!ModelState.IsValid) { TempData["Error"] = "Invalid data."; return RedirectToAction("Details", "Claim", new { id = vm.claimId }); } var log = new SettlementLog { claimId = vm.claimId, settlementAmount = vm.amount, remarks = vm.remarks ?? "" }; await _settlementService.approveSettlement(log); TempData["Success"] = $"Settlement approved for ${vm.amount}."; return RedirectToAction("Details", "Claim", new { id = vm.claimId }); } [HttpPost] [ValidateAntiForgeryToken] public async Task<IActionResult> RejectSettlement(int claimId, string remarks) { var log = new SettlementLog { claimId = claimId, settlementAmount = 0, remarks = string.IsNullOrEmpty(remarks) ? "No reason provided." : remarks }; await _settlementService.rejectSettlement(log); TempData["Warning"] = "Claim rejected."; return RedirectToAction("Details", "Claim", new { id = claimId }); } [HttpPost] [ValidateAntiForgeryToken] public async Task<IActionResult> ProcessPayment(int settlementId, int claimId) { bool success = await _settlementService.processPayment(settlementId); if (success) TempData["Success"] = "Payment processed successfully!"; else TempData["Error"] = "Payment failed."; return RedirectToAction("Details", "Claim", new { id = claimId }); } }
==========
========== FILE: Views/_ViewImports.cshtml
==========
========== @using InsuranceClaimsProcessingSystem @using InsuranceClaimsProcessingSystem.Models @using InsuranceClaimsProcessingSystem.Models.Enums @using InsuranceClaimsProcessingSystem.Models.ViewModels @addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
==========
========== FILE: Views/Shared/_Layout.cshtml

```
================================================================================ <!DOCTYPE html> <html lang="en"> <head> <meta charset="utf-8" /> <meta name="viewport" content="width=device-width, initial-scale=1.0" /> <title>@ViewData["Title"] - Insurance Claims System</title> <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet"> <style> .status-badge { font-size: 0.9rem; padding: 6px 12px; } .card-module { margin-bottom: 1.5rem; border-left: 4px solid #0d6efd; } .card-header-custom { background-color: #f8f9fa; border-bottom: 1px solid #dee2e6; font-weight: 600; } .fraud-high { background-color: #dc3545 !important; color: white; } .fraud-medium { background-color: #ffc107 !important; color: black; } .fraud-low { background-color: #198754 !important; color: white; } </style> </head> <body> <nav class="navbar navbar-expand-lg navbar-dark bg-primary shadow-sm mb-4"> <div class="container"> <a class="navbar-brand fw-bold" asp-controller="Claim" asp-action="Index">Claims Processing System</a> <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navContent"> <span class="navbar-toggler-icon"></span> </button> <div class="collapse navbar-collapse" id="navContent"> <ul class="navbar-nav me-auto mb-2 mb-lg-0"> <li class="nav-item"> <a class="nav-link active" asp-controller="Claim" asp-action="Index">All Claims</a> </li> @if (User.IsInRole("Customer") || User.IsInRole("ClaimsOfficer")) { <li class="nav-item"> <a class="nav-link" asp-controller="Claim" asp-action="RegisterClaim">+ New Claim</a> </li> } </ul> <div class="d-flex align-items-center gap-2 text-white"> <span>Hello, @User.Identity?.Name!</span> <form method="post" asp-controller="Account" asp-action="Logout" class="d-inline"> <button type="submit" class="btn btn-outline-light btn-sm">Logout</button> </form> </div> </div> </div> </nav> <div class="container-fluid px-4"> @if (TempData["Success"] != null) { <div class="alert alert-success alert-dismissible fade show" role="alert"> @TempData["Success"]<button type="button" class="btn-close" data-bs-dismiss="alert"></button> </div> } @if (TempData["Error"] != null) { <div class="alert alert-danger alert-dismissible fade show" role="alert"> @TempData["Error"]<button type="button" class="btn-close" data-bs-dismiss="alert"></button> </div> } @if (TempData["Warning"] != null) { <div class="alert alert-warning alert-dismissible fade show" role="alert"> @TempData["Warning"]<button type="button" class="btn-close" data-bs-dismiss="alert"></button> </div> } @RenderBody() </div> <footer class="bg-light mt-5 py-4 text-center text-muted"> <div class="container"> <small>&amp;copy; 2024 Insurance Claims Processing &amp;amp; Settlement System | Module 3.1-3.5</small> </div> </footer> <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script> @RenderSection("Scripts", required: false) </body> </html>
================================================================================ FILE: Views/Claim/Index.cshtml
```

==========
@model List<InsuranceClaimsProcessingSystem.Models.Claim>
@{ ViewData["Title"] = "Claims Dashboard"; }

<h2 class="mb-0">Active Claims (@Model.Count)</h2>
@if (User.IsInRole("Customer") || User.IsInRole("ClaimsOfficer")) {
<a asp-action="RegisterClaim" class="btn btn-primary">+ Register New
}

<table class="table table-hover align-middle mb-0">
<thead class="table-dark">
<tr>
<th>Claim ID</th>
<th>Policy #</th>
<th>Customer ID</th>
<th>Type</th>
<th>Amount ($)</th>
<th>Current Status</th>
<th style="width:200px">Operations</th>
</tr>
</thead>
<tbody>
@foreach (var item in Model) {
<tr>
<td>#@item.claimId</td>
<td>@item.policyId</td>
<td>@item.customerId</td>
<td>@item.claimType</td>
<td>@item.claimAmount.ToString("N2")</td>
<td>
@switch(item.claimStatus) {
case ClaimStatus.REGISTERED:
@item.claimStatus; break;
case ClaimStatus.UNDER_REVIEW:
@item.claimStatus; break;
case ClaimStatus.APPROVED:
@item.claimStatus; break;
case ClaimStatus.REJECTED:
@item.claimStatus; break;
}
</td>
<td>

<a asp-action="Details" asp-route-id="@item.claimId" class="btn btn-outline-primary">Details
@if (item.claimStatus == ClaimStatus.REGISTERED && User.IsInRole("ClaimsOfficer")) {
<form asp-action="ValidatePolicy" asp-route-claimId="@item.claimId" method="post" class="d-inline">
<button type="submit" class="btn btn-outline-success">Validate</button>
</form>
}

</td>
</tr>
}
</tbody>
</table>

@if (!Model.Any()) {

<h4>No claims registered yet.</h4>
Click 'Register New' to start the process.

}
==========
FILE: Views/Claim/RegisterClaim.cshtml
==========
@model InsuranceClaimsProcessingSystem.Models.Claim
@{ ViewData["Title"] = "Register New Claim"; }
<h2 class="mb-4">New Claim Registration (Module 3.1)</h2>

<form asp-action="RegisterClaim" method="post">

<label asp-for="policyId" class="form-label fw-bold">Policy ID *</label>
<input asp-for="policyId" class="form-control form-

control-lg" required /> <label asp-for="customerId" class="form-label fw-bold">Customer ID *</label> <input asp-for="customerId" class="form-control form-control-lg" required /> <label asp-for="claimType" class="form-label fw-bold">Claim Type *</label> <select asp-for="claimType" class="form-select form-select-lg" required> <option value="">-- Select Type --</option> <option value="Motor">Motor Vehicle</option> <option value="Health">Health/Medical</option> <option value="Property">Property Damage</option> <option value="Life">Life Insurance</option> <option value="Travel">Travel</option> </select> <label asp-for="claimAmount" class="form-label fw-bold">Claim Amount (USD) *</label> $ <input asp-for="claimAmount" type="number" step="0.01" min="0" class="form-control" required /> <button type="submit" class="btn btn-primary btn-lg">Submit Registration</button> <a asp-action="Index" class="btn btn-outline-secondary">Cancel </form>
========== FILE: Views/Claim/Details.cshtml ==========
========== @model InsuranceClaimsProcessingSystem.Models.Claim @{ ViewData["Title"] = $"Claim #@Model.claimId - Full Workflow"; } <nav aria-label="breadcrumb" class="mb-3"> <ol class="breadcrumb"> <li class="breadcrumb-item"><a asp-action="Index">Claims <li class="breadcrumb-item active">Claim #@Model.claimId </nav> <div class="row g-4"> <div class="col-lg-4"> <div class="card card-module h-100"> <div class="card-header card-header-custom d-flex justify-content-between align-items-center"> Claim Overview @Model.claimId </div> <div class="card-body"> <dl class="row"> <dt class="col-5 text-muted">Policy ID:</dt> <dd class="col-7">@Model.policyId</dd> <dt class="col-5 text-muted">Customer:</dt> <dd class="col-7">#@Model.customerId</dd> <dt class="col-5 text-muted">Type:</dt> <dd class="col-7">@Model.claimType</dd> <dt class="col-5 text-muted">Requested Amt:</dt> <dd class="col-7"><h4 class="text-primary">$@Model.claimAmount.ToString("N2")</h4></dd> <dt class="col-5 text-muted">Status:</dt> <dd class="col-7"> @switch(Model.claimStatus){ case ClaimStatus.REGISTERED: REGISTERED; break; case ClaimStatus.UNDER_REVIEW: UNDER_REVIEW; break; case ClaimStatus.APPROVED: APPROVED; break; case ClaimStatus.REJECTED: <span class="badge

bg-danger">REJECTED; break; } </dd> </dl> <hr /> @if (Model.claimStatus == ClaimStatus.REGISTERED && User.IsInRole("ClaimsOfficer")) { <form asp-action="ValidatePolicy" asp-route-claimId="@Model.claimId" method="post"> <button type="submit" class="btn btn-success w-100 mb-2">Validate Policy</button> </form> } <a asp-action="Index" class="btn btn-outline-dark w-100">← Back to List Module 3.2: Document Submission & Verification <a asp-controller="ClaimDocument" asp-action="Upload" asp-route-claimId="@Model.claimId" class="btn btn-sm btn-primary">+ Upload New @if (Model.Documents != null && Model.Documents.Any()) { <table class="table table-sm table-hover"> <thead class="table-light"> <tr><th>ID</th><th>Name</th><th>OCR Status</th><th>File</th></tr> </thead> <tbody> @foreach(var doc in Model.Documents) { <tr> <td>@doc.documentId</td> <td>@doc.documentName</td> <td> @switch(doc.verificationStatus){ case VerificationStatus.VERIFIED: Verified; break; case VerificationStatus.PENDING: Pending OCR; break; case VerificationStatus.REJECTED: Rejected; break; } </td> <td><code class="small">@doc.filePath</code></td> </tr> } </tbody> </table> } else { <small>No documents attached.</small> } Module 3.3: Assessment & Surveyor @if (Model.Assessment != null) { Assigned Surveyor #@Model.Assessment.surveyorId Assessed Value <h5 class="text-success">$@Model.Assessment.assessedAmount.ToString("N2")</h5> Remarks <small>@Model.Assessment.assessmentRemarks</small> } else { No surveyor assigned or assessment recorded. <a asp-controller="Assessment" asp-action="AssignSurveyor" asp-route-claimId="@Model.claimId" class="btn btn-outline-primary btn-sm">Assign Surveyor <a asp-controller="Assessment" asp-action="RecordAssessment" asp-route-claimId="@Model.claimId" class="btn btn-outline-success btn-sm">Record Assessment } <div class="card card-module @(Model.FraudCheck != null ? "border-danger" : "")"> Module 3.4: Fraud Detection @if(User.IsInRole("ComplianceOfficer") && Model.FraudCheck == null && Model.claimStatus != ClaimStatus.REGISTERED) { <form asp-controller="Fraud"

asp-action="RunFraudCheck" method="post" class="d-inline"> <input type="hidden" name="claimId" value="@Model.claimId" /> <button type="submit" class="btn btn-sm btn-outline-danger">Run Rules Engine</button> </form> } @if (Model.FraudCheck != null) { <div class="display-4 fw-bold text-@(Model.FraudCheck.riskFlag == RiskFlag.LOW ? "success" : Model.FraudCheck.riskFlag == RiskFlag.MEDIUM ? "warning" : "danger")"> @Model.FraudCheck.fraudScore <div class="fs-5 badge p-2 @(Model.FraudCheck.riskFlag == RiskFlag.LOW ? "bg-success text-white" : Model.FraudCheck.riskFlag == RiskFlag.MEDIUM ? "bg-warning text-black" : "bg-danger text-white")"> @Model.FraudCheck.riskFlag RISK <small class="d-block mt-1 text-muted">Rule-Based Scoring Result</small> <a asp-controller="Fraud" asp-action="GetFraudScore" asp-route-claimId="@Model.claimId" class="btn btn-sm btn-outline-secondary">View Report } else { <small>Fraud check pending execution</small> } Module 3.5: Settlement & Payment @if (Model.SettlementLogs != null && Model.SettlementLogs.Any()) { <table class="table table-sm table-bordered mb-3"> <thead class="table-light"> <tr><th>ID</th><th>Status</th><th>Amt</th><th>Notes</th></tr> </thead> <tbody> @foreach(var log in Model.SettlementLogs.OrderByDescending(x => x.settlementId)) { <tr> <td>#@log.settlementId</td> <td> @log.settlementStatus </td> <td>$@log.settlementAmount.ToString("N2")</td> <td><small>@log.remarks</small></td> </tr> } </tbody> </table> } @{ var lastLog = Model.SettlementLogs?.OrderByDescending(l => l.settlementId).FirstOrDefault(); bool hasApproved = lastLog != null && lastLog.settlementStatus == "APPROVED"; bool isPaid = lastLog != null && lastLog.settlementStatus == "PAYMENT_PROCESSED"; } @if ((Model.claimStatus == ClaimStatus.UNDER_REVIEW || Model.claimStatus == ClaimStatus.APPROVED) && User.IsInRole("ClaimsOfficer")) { <h6 class="fw-bold mb-3">Quick Actions</h6> @if (!hasApproved && !isPaid && Model.claimStatus == ClaimStatus.UNDER_REVIEW) { <form asp-controller="Settlement" asp-action="ApproveSettlement" method="post" class="mb-2"> <input type="hidden" name="claimId" value="@Model.claimId" /> <label class="form-label form-label-sm">Amount $</label> <input type="number" step="0.01" name="amount" class="form-control form-control-sm" value="@Model.claimAmount" required /> <label class="form-label form-label-sm">Notes</label> <input type="text" name="remarks"

class="form-control form-control-sm" placeholder="Optional" /> <button type="submit" class="btn btn-success btn-sm">Approve</button> </form> <form asp-controller="Settlement" asp-action="RejectSettlement" method="post" class="mb-2"> <input type="hidden" name="claimId" value="@Model.claimId" /> <input type="text" name="remarks" class="form-control form-control-sm" placeholder="Rejection reason..." required /> <button type="submit" class="btn btn-danger btn-sm w-100">Reject</button> </form> } @if (hasApproved && !isPaid) { <form asp-controller="Settlement" asp-action="ProcessPayment" method="post" class="d-inline"> <input type="hidden" name="settlementId" value="@lastLog!.settlementId" /> <input type="hidden" name="claimId" value="@Model.claimId" /> <button type="submit" class="btn btn-primary btn-sm">Process Payment ($@lastLog.settlementAmount.ToString("N2"))</button> </form> } @if (isPaid) { Payment Completed. } }
==========
========== FILE: Views/ClaimDocument/Upload.cshtml
==========
========== @model InsuranceClaimsProcessingSystem.Models.ViewModels.ClaimDocumentUploadViewModel @{ ViewData["Title"] = "Upload Document"; } Module 3.2: Upload Document <form enctype="multipart/form-data" asp-action="UploadClaimDocument" method="post"> <input type="hidden" asp-for="claimId" /> <label asp-for="documentName" class="form-label">Document Name *</label> <input asp-for="documentName" class="form-control" required placeholder="e.g., Police Report" /> <label asp-for="documentFile" class="form-label">File Selection *</label> <input asp-for="documentFile" type="file" class="form-control" accept=".pdf,.jpg,.png" required /> <small class="text-muted">Max 10MB | PDF, JPG, PNG</small> <button type="submit" class="btn btn-primary w-100">Upload & Verify</button> <a asp-controller="Claim" asp-action="Details" asp-route-id="@Model.claimId" class="btn btn-outline-secondary">Cancel </form>
==========
========== FILE: Views/Assessment/AssignSurveyor.cshtml
==========
========== @model

InsuranceClaimsProcessingSystem.Models.ViewModels.AssignSurveyorViewModel @{ ViewData["Title"] = "Assign Surveyor"; } Module 3.3: Surveyor Assignment <form asp-action="AssignSurveyor" method="post"> <input type="hidden" asp-for="claimId" /> <label asp-for="surveyorId" class="form-label">Surveyor Employee ID *</label> <input asp-for="surveyorId" type="number" class="form-control form-control-lg" required /> <button type="submit" class="btn btn-primary btn-lg">Confirm Assignment</button> <a asp-controller="Claim" asp-action="Details" asp-route-id="@Model.claimId" class="btn btn-link text-decoration-none">Back to Claim </form>
========== FILE: Views/Assessment/RecordAssessment.cshtml ==========
========== @model InsuranceClaimsProcessingSystem.Models.ViewModels.RecordAssessmentViewModel @{ ViewData["Title"] = "Record Assessment"; } Module 3.3: Record Loss Assessment <form asp-action="RecordAssessment" method="post"> <input type="hidden" asp-for="claimId" /> <label asp-for="surveyorId" class="form-label">Surveyor ID *</label> <input asp-for="surveyorId" type="number" class="form-control" required /> <label asp-for="assessedAmount" class="form-label">Assessed Amount ($)</label> <div class="input-group"> $ <input asp-for="assessedAmount" type="number" step="0.01" class="form-control" required /> <label asp-for="assessmentRemarks" class="form-label">Observations</label> <textarea asp-for="assessmentRemarks" class="form-control" rows="4"></textarea> <button type="submit" class="btn btn-success">Save Assessment</button> <a asp-controller="Claim" asp-action="Details" asp-route-id="@Model.claimId" class="btn btn-outline-secondary">Cancel </form>
========== FILE: Views/Fraud/Details.cshtml ==========
========== @model InsuranceClaimsProcessingSystem.Models.FraudCheck @{ ViewData["Title"] = "Fraud Analysis Report"; } <div class="card shadow-lg

overflow-hidden"> <div class="card-header bg-@(Model.riskFlag == RiskFlag.HIGH ? "danger" : Model.riskFlag == RiskFlag.MEDIUM ? "warning" : "primary") text-white"> <h3 class="mb-0">Fraud Detection Report</h3> <div class="display-1 fw-bold text-@(Model.riskFlag == RiskFlag.HIGH ? "danger" : Model.riskFlag == RiskFlag.MEDIUM ? "warning" : "success")"> @Model.fraudScore @Model.riskFlag RISK Claim Ref #@Model.claimId Engine Used Rule-Based v1.0 Timestamp @DateTime.Now.ToString("yyyy-MM-dd HH:mm") <a asp-controller="Claim" asp-action="Details" asp-route-id="@Model.claimId" class="btn btn-primary btn-lg">Return to Claim Workflow
==========
========== FILE: Program.cs
==========
========== using InsuranceClaimsProcessingSystem.Data; using InsuranceClaimsProcessingSystem.Services; using InsuranceClaimsProcessingSystem.Services.Interfaces; using Microsoft.AspNetCore.Identity; using Microsoft.EntityFrameworkCore; var builder = WebApplication.CreateBuilder(args); builder.Services.AddDbContext<ApplicationDbContext>(options => options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"))); builder.Services.AddIdentity<IdentityUser, IdentityRole>(options => { options.Password.RequireDigit = true; options.Password.RequiredLength = 6; options.Password.RequireNonAlphanumeric = false; options.Password.RequireUppercase = false; }) .AddEntityFrameworkStores<ApplicationDbContext>() .AddDefaultTokenProviders(); builder.Services.AddScoped<IClaimService, ClaimService>(); builder.Services.AddScoped<IClaimDocumentService, ClaimDocumentService>(); builder.Services.AddScoped<IAssessmentService, AssessmentService>(); builder.Services.AddScoped<IFraudService, FraudService>(); builder.Services.AddScoped<ISettlementService, SettlementService>(); builder.Services.AddControllersWithViews(); var app = builder.Build(); if (!app.Environment.IsDevelopment()) { app.UseExceptionHandler("/Home/Error"); app.UseHsts(); } app.UseHttpsRedirection(); app.UseStaticFiles(); app.UseRouting(); app.UseAuthentication(); app.UseAuthorization(); app.MapControllerRoute(name: "default", pattern: "{controller=Claim}/{action=Index}/{id?}"); using (var scope =

app.Services.CreateScope()) { var services = scope.ServiceProvider; var roleManager = services.GetRequiredService<RoleManager<IdentityRole>>(); string[] roles = { "Customer", "ClaimsOfficer", "Surveyor", "ComplianceOfficer", "Admin" }; foreach (var role in roles) { if (!await roleManager.RoleExistsAsync(role)) await roleManager.CreateAsync(new IdentityRole(role)); } } app.Run();
==========
========== FILE: appsettings.json
==========
========== { "Logging": { "LogLevel": { "Default": "Information", "Microsoft.AspNetCore": "Warning" } }, "AllowedHosts": "*", "ConnectionStrings": { "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=InsuranceClaimsDb;Trusted_Connection=True; TrustServerCertificate=True;" } }
==========
========== FILE: schema.sql
==========
========== CREATE TABLE IF NOT EXISTS Customer (customerId INT IDENTITY(1,1) PRIMARY KEY, customerName VARCHAR(100) NOT NULL, email VARCHAR(100), phone VARCHAR(20)); CREATE TABLE IF NOT EXISTS Policy (policyId INT IDENTITY(1,1) PRIMARY KEY, policyNumber VARCHAR(100) NOT NULL UNIQUE, coverageAmount DECIMAL(15,2) NOT NULL, isActive BIT DEFAULT 1, customerId INT, FOREIGN KEY (customerId) REFERENCES Customer(customerId)); CREATE TABLE IF NOT EXISTS Claim (claimId INT IDENTITY(1,1) PRIMARY KEY, policyId INT NOT NULL, customerId INT NOT NULL, claimType VARCHAR(50) NOT NULL, claimAmount DECIMAL(15,2) NOT NULL, claimStatus VARCHAR(20) DEFAULT 'REGISTERED', FOREIGN KEY (policyId) REFERENCES Policy(policyId), FOREIGN KEY (customerId) REFERENCES Customer(customerId)); CREATE TABLE IF NOT EXISTS ClaimDocument (documentId INT IDENTITY(1,1) PRIMARY KEY, claimId INT NOT NULL, documentName VARCHAR(100) NOT NULL, filePath VARCHAR(255) NOT NULL, verificationStatus VARCHAR(20) DEFAULT 'PENDING', FOREIGN KEY (claimId) REFERENCES Claim(claimId) ON DELETE CASCADE); CREATE TABLE IF NOT EXISTS Assessment (assessmentId INT IDENTITY(1,1) PRIMARY KEY, claimId INT NOT NULL UNIQUE, surveyorId INT NOT NULL, assessedAmount DECIMAL(15,2) DEFAULT 0.00, assessmentRemarks VARCHAR(255) DEFAULT '', FOREIGN KEY (claimId) REFERENCES Claim(claimId)); CREATE TABLE IF NOT EXISTS FraudCheck (fraudId INT IDENTITY(1,1) PRIMARY KEY, claimId INT NOT NULL UNIQUE, fraudScore INT DEFAULT 0, riskFlag VARCHAR(10), FOREIGN KEY (claimId) REFERENCES Claim(claimId)); CREATE TABLE IF NOT EXISTS SettlementLog (settlementId INT IDENTITY(1,1) PRIMARY KEY, claimId INT NOT NULL, settlementAmount DECIMAL(15,2) DEFAULT 0.00, settlementStatus VARCHAR(50) DEFAULT '', remarks VARCHAR(255) DEFAULT '', FOREIGN KEY (claimId) REFERENCES Claim(claimId));

==========
========== END OF SOURCE CODE
==========
========== ``` **How to save this as a PDF yourself:** 1. **Copy** all the text in the code block above. 2. **Paste** it into VS Code, Notepad++, or Word. 3. Go to **File → Print → Save as PDF** (or use an online Markdown-to-PDF converter). 4. Save the file as `InsuranceClaimsSystem_Complete.pdf`.